

PROJECT 1 - SCRIPT - CONTROLLED ACCESS CONTROL LIST (ACL): RESTRICTING RECORD ACCESS BASED ON FIELD VALUE IN SERVICENOW

ABSTRACT

Protecting sensitive records from unauthorized access is one of the most important requirements of any enterprise platform. ServiceNow provides a powerful security mechanism called the **Access Control List (ACL)**, which decides who can read, create, write and delete data at the table, record and field levels. This project, "**Script-Controlled ACL - Restrict Record Access Based on Field Value**", demonstrates how ACLs combined with roles, data conditions and server-side scripts can enforce record-level security in a ServiceNow instance.

A custom table, **Institution Details (u_institution_details)**, was created to store student information such as roll number, student name, faculty name, branch (ECE, EEE, CSE), email, phone number and description. Four custom roles - **bb1, bb2, bb3 and bb4** - were created and assigned to a test user named **EEE User**. Four ACLs were then configured on the table: a **Read ACL** (role bb1, data condition "Branch is EEE", with an advanced script that always allows the admin role), a **Create ACL** (role bb2), a **Write ACL** (role bb3) and a **Delete ACL** (role bb4).

The results were verified by impersonating different users. A user holding the bb1 role could view only the EEE branch records; a user without the role could not view any record; and an administrator could view all records regardless of branch. Users with bb2, bb3 and bb4 in addition could respectively create, edit and delete the EEE records. The project shows how the principle of **least privilege** can be implemented in ServiceNow by separating the permissions for each database operation and evaluating user roles together with record field values.

Keywords: ServiceNow, Access Control List (ACL), Role-Based Access Control, Record-Level Security, Data Condition, Script-Controlled ACL, Least Privilege.

1. INTRODUCTION

1.1 Overview

Modern enterprises store large volumes of business data on cloud platforms, and different users need different levels of access to that data. If every user can see and modify every record, the risk of data leakage and accidental damage becomes very high. **Access control** is therefore a core part of any information system.

ServiceNow is a cloud-based platform used for IT Service Management, HR, customer service and many other enterprise workflows. In ServiceNow, security is enforced through **Access Control Rules (ACLs)**. An ACL specifies the object being secured (table, record or field), the operation (read, write, create, delete), and the conditions that a user must satisfy - a required role, a data condition and, optionally, a script - before access is granted.

This project implements a **script-controlled ACL** on a custom table so that records are visible only when the user holds the required role and the record satisfies a field-value condition (Branch = EEE), while administrators retain full access.

1.2 About the Naan Mudhalvan Programme

Naan Mudhalvan is a flagship skilling initiative of the Government of Tamil Nadu, implemented by the Tamil Nadu Skill Development Corporation (TNSkills). It provides students of arts, science and engineering colleges with industry-aligned training and certification in emerging technologies so that they become employable. This project has been carried out as part of the ServiceNow course offered under the programme.

1.3 Objectives

- To create users, roles and a custom table in a ServiceNow instance.
- To restrict READ access to records based on the **Branch** field value and a custom role.
- To control **CREATE, WRITE and DELETE** operations through separate roles (bb2, bb3, bb4).
- To use a server-side ACL script that allows administrators full access.
- To verify the security rules by impersonating different users.

1.4 Scope of the Project

The project covers table-level record security for a single custom table, u_institution_details, in the Global application scope. It demonstrates the four basic operations (read, create, write,

delete). Field-level ACLs, Service Portal widgets and integration with external identity providers are outside the scope of this work.

2. SYSTEM STUDY

2.1 Problem Statement

In an educational institution, student records contain personal details such as e-mail addresses and phone numbers. Records of one department (for example EEE) should not be visible to users of other departments, and the rights to view, add, modify and delete records should be given separately. Without proper controls, any logged-in user could view or change the data of every branch.

2.2 Existing System

- By default, a newly created custom table may be readable by users who hold general roles, and access is not tied to the values stored in the record.
- Access is often managed only by assigning roles, with no record-level restriction.
- The same role frequently gives view, edit and delete rights together, which violates the principle of least privilege.

2.3 Proposed System

The proposed system defines a separate ACL for each operation on the `u_institution_details` table:

- **Read ACL** - requires role `bb1`, applies only to records where Branch is EEE, and runs an advanced script that returns true for admins and for `bb1` holders.
- **Create ACL** - requires role `bb2`.
- **Write ACL** - requires role `bb3`.
- **Delete ACL** - requires role `bb4`.

Because each permission is a separate role, an administrator can grant exactly the rights a user needs, and nothing more.

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Table 3.1: Hardware requirements

Component	Specification
Processor	Intel Core i3 or above (or equivalent)
RAM	4 GB minimum (8 GB recommended)
Storage	256 GB or above
Network	Stable internet connection

3.2 Software Requirements

Table 3.2: Software requirements

Component	Specification
Platform	ServiceNow Personal Developer Instance (PDI) / training instance
Access level	Administrator (admin) with the security_admin role elevated
Web browser	Google Chrome / Microsoft Edge (latest version)
Scripting language	JavaScript (server-side ACL script)
Application scope	Global

4. SYSTEM DESIGN

4.1 How ACL Evaluation Works

When a user tries to perform an operation on a record, ServiceNow evaluates the matching ACLs. Access is granted only when **all** of the following pass for the ACL:

1. The user holds at least one of the roles listed in the **Requires role** related list.
2. The record satisfies the **Data Condition**, if one is defined.
3. The **Advanced script**, if enabled, returns true.

If no ACL grants access, the operation is denied. Users with the admin role can bypass ACLs that have the "Admin overrides" option selected.

4.2 Database Design

The custom table **Institution Details** (name: u_institution_details) has been created with "Extends table" left empty. Its fields are shown below.

Table 4.1: Fields of the u_institution_details table

Field (Column label)	Type	Remarks
Student Roll Number	Auto Number	Generated automatically (e.g., INS0001007)
Student Name	Reference (User)	Refers to the sys_user table
Faculty Name	Reference (User)	Refers to the sys_user table
Branch	Choice	ECE, EEE, CSE - used in the data condition
Email	String	Student e-mail address
Phone Number	String	Student contact number
Description	Multi-line String	Free-text remarks; edited to test the Write ACL

4.3 Roles Design

Table 4.2: Custom roles and their purpose

Role	Operation	Purpose
bb1	Read	Allows viewing of EEE branch records
bb2	Create	Allows creating new records (New button visible)
bb3	Write	Allows editing of existing records
bb4	Delete	Allows deleting records

4.4 ACL Design

Table 4.3: ACL configuration summary

ACL	Operation	Required role	Advanced	Data condition / notes
Read	read	bb1	Yes	Branch is EEE; script allows admin and bb1
Create	create	bb2	No	No data condition
Write	write	bb3	No	No data condition
Delete	delete	bb4	No	No data condition

4.5 Access Matrix

Table 4.4: Expected access for each type of user

User / roles	View	Create	Edit	Delete
Admin	All records	Yes	Yes	Yes
User without roles	None	No	No	No
bb1	EEE only	No	No	No
bb1 + bb2	EEE only	Yes	No	No
bb1 + bb2 + bb3	EEE only	Yes	Yes	No
bb1 + bb2 + bb3 + bb4	EEE only	Yes	Yes	Yes

5. IMPLEMENTATION

The project was implemented in milestones. Each step below was carried out in the ServiceNow instance with the administrator account.

5.1 Milestone 1 - Creation of Users and Roles

5.1.1 User creation

1. Log in to the ServiceNow instance with admin access.
2. Navigate to **User Administration > Users** and click **New**.
3. Enter User ID: **EEE User**, First name: **EEE**, Last name: **User**, Email: **eeeuser@gmail.com**.
4. Save the record.

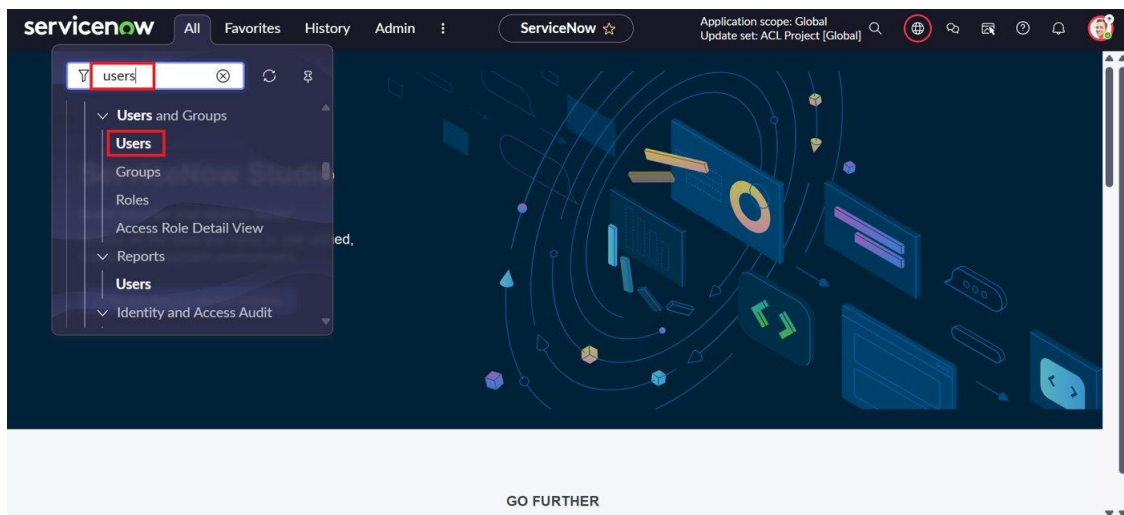


Figure 1: Navigating to Users in the Application Navigator

A screenshot of the ServiceNow 'User' form for 'EEE User'. The form is divided into two columns. The left column contains fields for 'User ID' (EEEUser), 'First name' (EEE), 'Last name' (User), 'Title', 'Department', and 'Location'. The right column contains fields for 'Email' (eeeuser@gmail.com), 'Identity type' (Human), 'Language' (-- None --), 'Calendar integration' (Outlook), 'Time zone' (System (Asia/Kolkata)), 'Date format' (System (yyyy-MM-dd)), 'Business phone', and 'Mobile phone'. At the bottom, there are checkboxes for 'Password needs reset', 'Locked out', 'Active' (checked), and 'Internal Integration User'. The 'Update' button is highlighted with a red box.

Figure 2: Creating the test user "EEE User"

5.1.2 Role creation

1. Navigate to **User Administration > Roles** and click **New**.

2. Enter the role name **bb1**, add a brief description and click **Submit**.
3. Repeat the same procedure to create the roles **bb2**, **bb3** and **bb4**.
4. Assign all four roles to the **EEE User**.

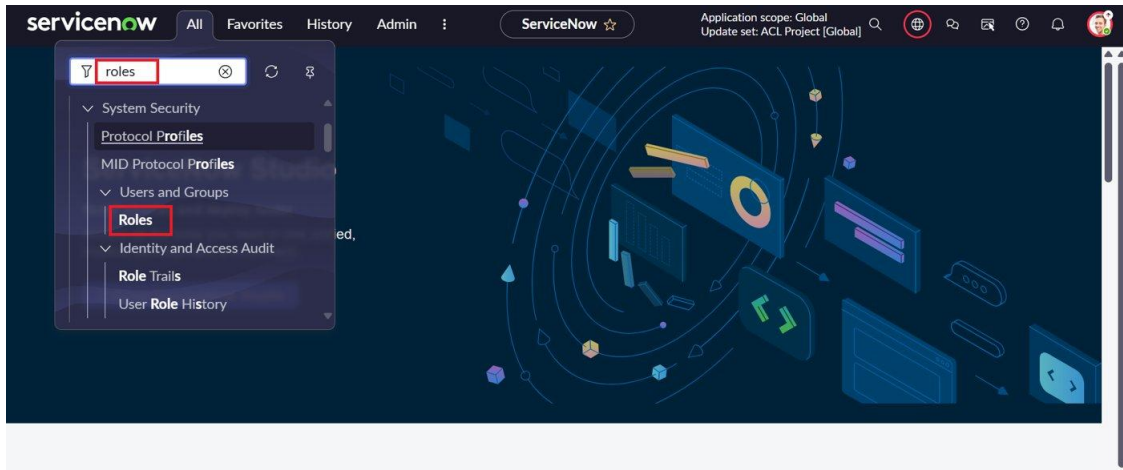


Figure 3: Navigating to Roles in the Application Navigator

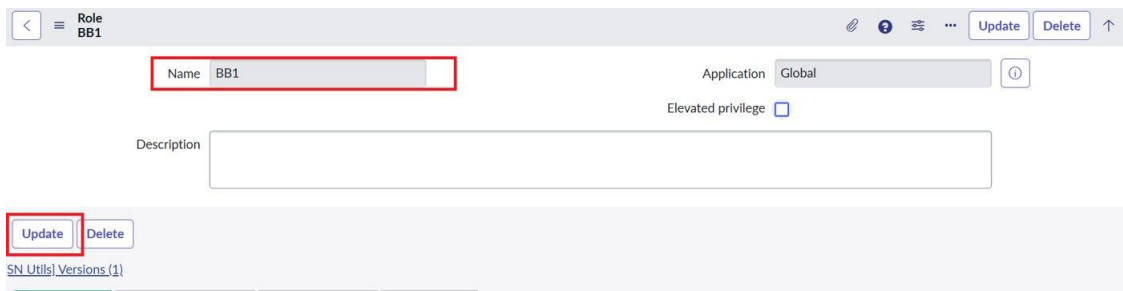


Figure 4: Creating the custom role BB1

5.2 Milestone 2 - Table Creation

1. In the Application Navigator, search for **System Definition > Tables** and click **New**.
2. Enter Label: **Institution Details**, Name: **u_institution_details**, and leave "Extends table" empty. Click **Submit**.
3. Add the fields: Student Roll Number (Auto Number), Student Name (Reference - User), Faculty Name (Reference - User), Branch (Choice: ECE, EEE, CSE), Email (String), Phone Number (String) and Description (Multi-line String).
4. Save the table and create multiple records with different branch values (ECE, EEE, CSE).

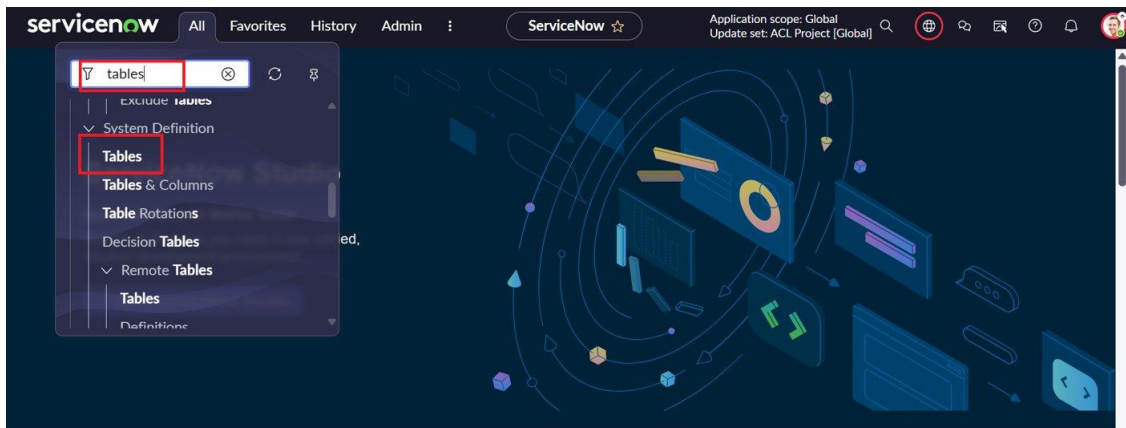


Figure 5: Navigating to System Definition > Tables

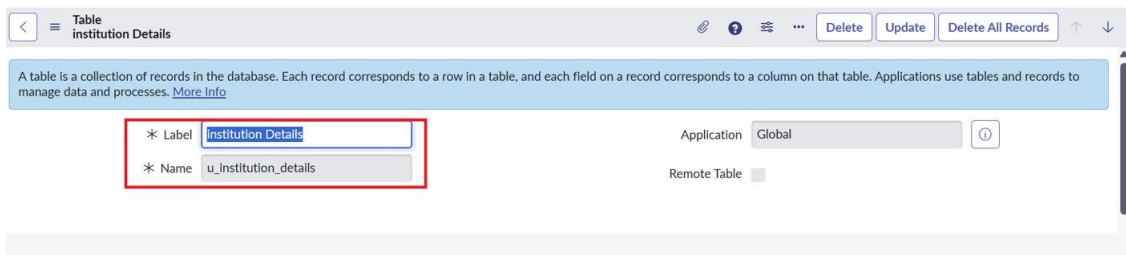


Figure 6: Creating the custom table Institution Details (u_institution_details)

Column label	Type	Reference	Max length	Default value
Updates	Integer	(empty)	40	
Created by	String	(empty)	40	
Student Roll Number	String	(empty)	40	javascript:global.getNextObjNumberPadded();
Description	String	(empty)	1,000	
Updated by	String	(empty)	40	
Phone Number	String	(empty)	40	
Sys ID	Sys ID (GUID)	(empty)	32	
Created	Date/Time	(empty)	40	
Faculty name	Reference	User	32	
Branch	String	(empty)	40	
Updated	Date/Time	(empty)	40	
Student Name	Reference	User	32	
Email	String	(empty)	40	

Figure 7: Table columns (dictionary entries) of u_institution_details

institution Details

Student Roll Numbe

Search

Actions on selected rows...

All

Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
Search	Search	Search	Search	Search	Search
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	saiguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

Figure 8: Sample records created with ECE, EEE and CSE branch values

5.3 Milestone 3 - Creation of the Read ACL

1. Elevate the **security_admin** role from the user menu (profile icon) by choosing **Elevate role**, selecting **security_admin** and clicking **Update**.
2. In the Application Navigator, search for **System Security > Access Control (ACL)** and click **New**.
3. Enter Type: **record**, Operation: **read**, Name: **u_institution_details**, Active: **true**, and tick **Advanced**.
4. In the **Requires role** related list, add the role **bb1**.
5. Under **Data Condition**, add the condition **Branch is EEE**.
6. In the **Script** field, add the script given in Appendix A, and click **Submit**.

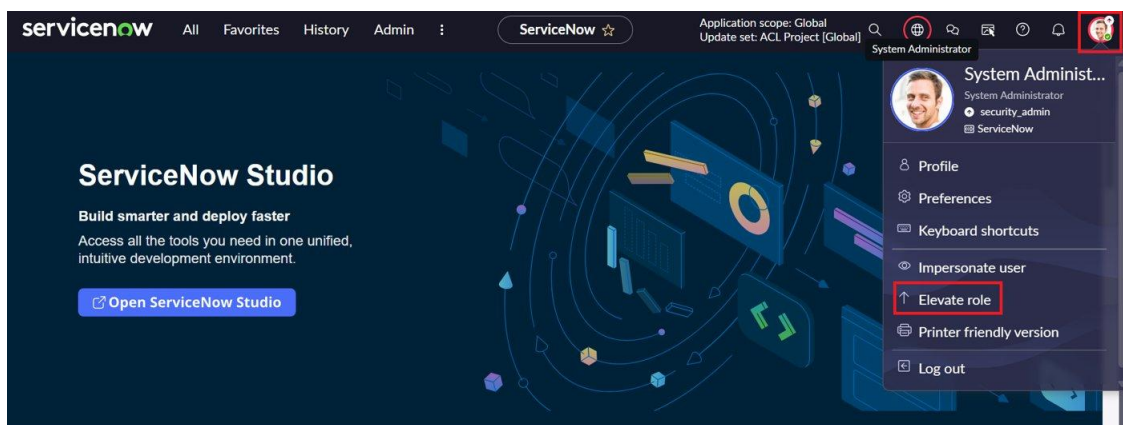


Figure 9: Elevating the role from the user menu

Elevate role

Close dialog

Elevate a role by adding privileges, which end when you log out. [Learn more](#)

AVAILABLE ROLES

☒ security_admin

Grant modification access to High Security Settings, allow user to modify the Access Control List

Cancel

Update

Figure 10: Selecting security_admin in the Elevate role dialog

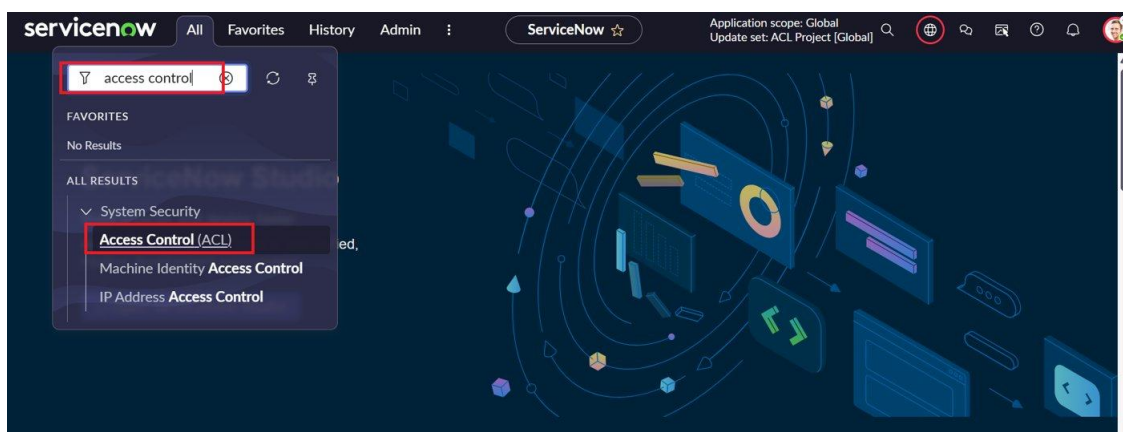


Figure 11: Opening System Security > Access Control (ACL)

Access Controls							
Updated							
Search							
All							
Search							
Name							
Decision Type							
Operation							
Type							
Active							
Updated by							
Updated							
sn_try_build_agent_message							
sn_try_build_agent_freemium_usage_log							
sn_try_build_agent_conversation							
sn_try_build_agent_conversation							
sn_try_build_agent_message							
sn_try_build_agent_event_telemetry							
sn_try_build_agent_conversation							
sn_try_build_agent_task_telemetry							
sn_try_build_agent_task_telemetry							
sn_try_build_agent_event_telemetry							
sn_try_build_agent_task_telemetry							
sn_try_build_agent_knowledge_source							
sn_try_build_agent_event_telemetry							

Figure 12: Access Controls list with the New button

Access Control
u_institution_details

* Type: record

* Operation: read

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: institution Details [u_institution_details]

Description: </>

Applies To: No. of records matching the condition: 6

Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

Application: Global

Active: ☒

Advanced: ☒

Figure 13: Read ACL - Type, Operation, Name and Advanced option

Data Condition

Condition: No. of records matching the condition: 2

Add Filter Condition Add OR Clause

Branch is EEE

AND OR X

Advanced Condition

Controlled by References: ☐

Script: Turn on ECMAScript 2021 (ES12) mode

```

1
2
3
4
5
6
7
8
(function() {
  if (gs.hasRole('bb1')) {
    return true;
  } else {
    return false;
  }
})();

```

Figure 14: Read ACL - Data Condition (Branch is EEE) and Advanced Condition script

Script logic: The script first checks whether the current user has the admin role and, if so, returns true so that administrators always retain full access. It then checks for the bb1 role and returns true for its holders. In every other case it returns false, so access is denied.

Testing the Read ACL: Impersonate the EEE user and open the list by typing **u_institution_details.list** in the All menu. Then impersonate the admin and open the same list.

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
INS0001007	EEE		saiguna@gmail.com	Abel Tuter	9090898765	Guna Sai
INS0001004	EEE		prashanth@gmail.com	Abel Tuter	9090909090	Prashanth Reddy

Figure 15: Output: user with the bb1 role sees only EEE branch records

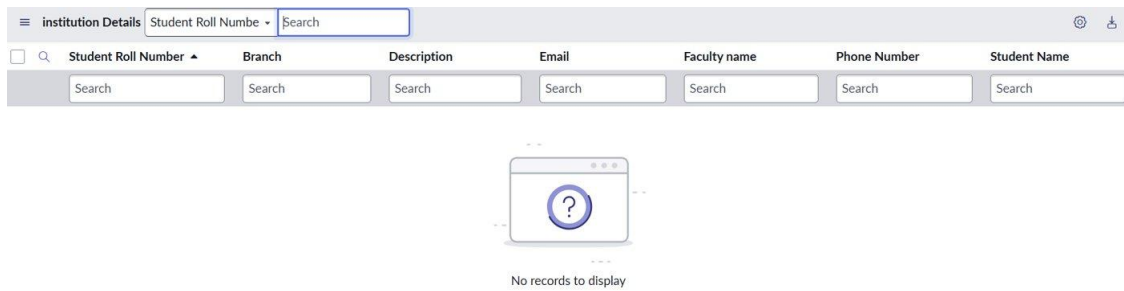


Figure 16: Output: user without the role sees no records

Student Roll Number	Faculty name	Student Name	Branch	Email	Phone Number
INS0001006	ITIL User	Praneeth	ECE	praneeth@gmail.com	8989090906
INS0001008	Malla Sravan kumar	Abraham Lincoln	CSE	sravan@gmail.com	8978908989
INS0001005	ITIL User	Srinu Nandipalli	ECE	srinu@gmail.com	9089909090
INS0001007	Abel Tuter	Guna Sai	EEE	saiguna@gmail.com	9090898765
INS0001001	Abraham Lincoln	Yaswanth Nikku	CSE	yaswanth@gmail.com	8989898989
INS0001004	Abel Tuter	Prashanth Reddy	EEE	prashanth@gmail.com	9090909090

Figure 17: Output: admin user sees all records regardless of branch

5.4 Milestone 4 - Creation of the Create ACL

1. Click **New** in the Access Controls list.
2. Enter Type: **record**, Operation: **create**, Name: **u_institution_details**, Active: **true**.
3. In the **Requires role** related list, add **bb2**. No data condition is required.
4. Save the ACL record.

Figure 18: Create ACL - Operation set to create

Conditions

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

[More Info](#)

Requires role

Role
BB2
Insert a new row...

Security Attribute Condition

Figure 19: Create ACL - Requires role BB2

Verification: A user holding both bb1 and bb2 can view only the EEE branch records and now also sees the **New** button.

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
INS0001007	EEE		saiguna@gmail.com	Abel Tuter	9090898765	Guna Sai
INS0001004	EEE		prashanth@gmail.com	Abel Tuter	9090909090	Prashanth Reddy

Figure 20: Output: user with bb1 and bb2 sees EEE records with the New button

5.5 Milestone 5 - Creation of the Write ACL

1. Click **New** to create another ACL.
2. Enter Type: **record**, Operation: **write**, Name: **u_institution_details**, Active: **true**.
3. In the **Requires role** related list, add **bb3**. No data condition is required.
4. Save the ACL record.

Access Control

u_institution_details

* Type: record

* Operation: write

Decision Type: Allow If

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description:

Applies To: [No. of records matching the condition: 6](#)

[Add Filter Condition](#) [Add OR Clause](#)

-- choose field -- -- oper -- -- value --

Figure 21: Write ACL - Operation set to write

Access Control Rules have two decision types, and these types will behave differently depending on conditions.

1. Allow Access: Allows access to a resource if all conditions are met. Additional ACLs may grant access to records where this ACL has not granted access.
2. Deny Access: Denies access to a resource unless all conditions are met. Additional ACLs may not grant access to records where this ACL has denied access.

[More Info](#)

Requires role

Role
BB3

+

Insert a new row...

1 to 1 of 1

Figure 22: Write ACL - Requires role BB3

Verification: A user holding bb1, bb2 and bb3 can view and edit the EEE branch records. The Description field was edited inline and saved as "ACL is Executed".

institution Details

Student Roll Number

Search

Actions on selected rows...

New

All

☐	Student Roll Number	Branch	Description	Email	Faculty name	Phone Number	Student Name
	INS0001007	EEE		com	Abel Tuter	9090898765	Guna Sai
	INS0001004	EEE		ill.com	Abel Tuter	9090909090	Prashanth Reddy

Figure 23: Inline editing of the Description field

institution Details

Student Roll Number

Search

Actions on selected rows...

New

All

Student Roll Number

Branch

Description

Email

Faculty name

Phone Number

Student Name

INS0001007

EEE

ACL is Executed

saiguna@gmail.com

Abel Tuter

9090898765

Guna Sai

INS0001004

EEE

prashanth@gmail.com

Abel Tuter

9090909090

Prashanth Reddy

Figure 24: Output: Description updated ("ACL is Executed") by user with bb1, bb2 and bb3

5.6 Milestone 6 - Creation of the Delete ACL

1. Click **New** to create another ACL.
2. Enter Type: **record**, Operation: **delete**, Name: **u_institution_details**, Active: **true**.
3. In the **Requires role** related list, add **bb4**. No data condition is required.
4. Save the ACL record.

Warning: A role, security attribute, data condition, script or ACL control via reference fields is required to properly secure access with this ACL.

* Type: record

* Operation: delete

Decision Type: Allow If

Application: Global

Active: ☒

Advanced: ☐

Admin overrides: ☒

Protection policy: -- None --

* Name: Institution Details [u_institution_details]

Description: </>

Applies To: No. of records matching the condition: 6

Add Filter Condition Add OR Clause

-- choose field -- -- oper -- -- value --

Figure 25: Delete ACL - Operation set to delete

Requires role

Role
BB4

Insert a new row...

Security Attribute Condition

Figure 26: Delete ACL - Requires role BB4

Verification: A user holding bb1, bb2, bb3 and bb4 can view, create, edit and delete the EEE branch records. Selecting a record and choosing **Delete** from "Actions on selected rows" displays a confirmation dialog.

Student Roll Number	Branch	Description	Email	Faculty name	Phone Number
INS0001007	EEE	ACL is Executed	saiguna@gmail.com	Abel Tuter	90908987
INS0001004	EEE	ACL is Executed	prashanth@gmail.com	Abel Tuter	90909090

Actions on selected rows...

- Delete
- Delete with preview...
- Create Application File
- Assign Tag:
 - New tag
 - Now Platform Security
 - Strategic Portfolio Management
 - Field Service Management
 - Now Platform
 - Security
 - IT Asset Management
 - IT Service Management
 - IT Operations Management
 - More...

Figure 27: Delete option under "Actions on selected rows"

Confirmation

Delete the selected record(s)?

Cancel Delete

Figure 28: Delete confirmation dialog

6. TESTING AND RESULTS

6.1 Test Cases

Table 6.1: Test cases and observed results

S.No	User / roles	Action	Result
1	Admin	Open u_institution_details.list	All records displayed (ECE, EEE, CSE) - Figure 17
2	User with bb1	Open the list	Only EEE records displayed - Figure 15
3	User without the role	Open the list	No records to display - Figure 16
4	User with bb1 + bb2	Open the list	EEE records and New button visible - Figure 20
5	User with bb1 + bb2 + bb3	Edit Description inline	Value saved as "ACL is Executed" - Figure 24
6	User with bb1 + bb2 + bb3 + bb4	Select a record and choose Delete	Delete option and confirmation shown - Figures 27, 28

6.2 Result Analysis

The results match the expected access matrix (Table 4.4). Record-level visibility was determined by both the user's role and the Branch value of the record, and each additional operation became available only when its corresponding role was assigned. Administrators were unaffected by the restrictions, as intended.

7. ADVANTAGES AND LIMITATIONS

7.1 Advantages

- Provides fine-grained, record-level security based on field values.
- Separates read, create, write and delete permissions, following the principle of least privilege.
- Administrators retain full control and can manage access easily through roles.
- The same approach can be reused for other tables and departments.
- Implemented entirely with built-in platform features and a short script.

7.2 Limitations

- The data condition in this project is fixed to the EEE branch; other branches would need their own ACLs and roles.
- Create, Write and Delete ACLs are role-based only and do not use data conditions.
- Roles must be assigned manually to each user.
- Field-level ACLs were not implemented, so all fields of a visible record are accessible.

8. CONCLUSION

This project successfully demonstrated how script-controlled ACLs enforce record-level security in ServiceNow. By combining custom roles, a data condition on the Branch field and a server-side script, the system ensured that only authorised users could view, create, edit or delete records, while administrators kept complete access. The verification with different impersonated users confirmed that the ACLs work as designed.

Through this work, I gained practical understanding of users, roles, custom tables, ACL evaluation, elevated roles (security_admin) and user impersonation. These are essential skills for a ServiceNow administrator or developer, and the project has strengthened my readiness for entry-level roles in the IT industry.

9. FUTURE SCOPE

- Create separate read ACLs and roles for the ECE and CSE branches.
- Add field-level ACLs to hide sensitive fields such as e-mail and phone number.
- Use user groups instead of assigning roles individually.
- Restrict faculty so that they can view only the students assigned to them, using an advanced script.
- Expose the table through a Service Portal page and confirm that the same ACLs apply.
- Add audit and logging to track who accessed or changed a record.

REFERENCES

1. ServiceNow Documentation - Access Control Rules. <https://docs.servicenow.com>
2. ServiceNow Developer Site - Personal Developer Instance.
<https://developer.servicenow.com>
3. ServiceNow Community - Security and ACL discussions.
<https://www.servicenow.com/community>
4. Naan Mudhalvan (TNSkills) - ServiceNow course material.
<https://naanmudhalvan.tn.gov.in>

APPENDIX A - READ ACL SCRIPT

The following advanced script was added to the Read ACL of the `u_institution_details` table:

```
(function () {  
  
    // Allow admin users full access  
    if (gs.hasRole('admin')) {  
        return true;  
    }  
  
    // Allow only EEE branch users (role bb1) to see EEE records  
    if (gs.hasRole('bb1')) {  
        return true;  
    }  
  
    // Deny access for all others  
    return false;  
  
})();
```

APPENDIX B - ACL CONFIGURATION SUMMARY

Table B.1: ACL fields used in the project

Field	Value
Type	record
Name	Institution Details [<code>u_institution_details</code>]
Decision type	Allow If
Admin overrides	Selected
Active	true
Operations configured	read (bb1, Advanced), create (bb2), write (bb3), delete (bb4)
Data condition (Read ACL)	Branch is EEE